

GUÍA OPERACIONAL: CRYPTOCURRENCY EXCHANGE QA FRAMEWORK

Setup Local | GitHub Actions | Mantenimiento | Debugging | Troubleshooting

TABLA DE CONTENIDOS

- 1. Setup e Instalaci3n Local
- 2. Ejecutar Tests Localmente
- 3. Entender los Resultados
- 4. GitHub Actions Pipelines
- 5. Debugging y Troubleshooting
- 6. Ver Reportes y Bugs
- 7. Mantenimiento del Framework
- 8. Best Practices
- 9. FAQ

1. SETUP E INSTALACION LOCAL

1.1 Prerequisites (Requisitos)

Verifica que tengas instalado:

Java JDK 11+
java -version
Debe mostrar: openjdk version "11.x.x" o superior

Maven 3.8+
mvn -version
Debe mostrar: Apache Maven 3.8.x o superior

Git
git --version
Debe mostrar: git version 2.x.x o superior

Chrome o Firefox (para Selenium)
Si usas Chrome: descarga de <https://chromedriver.chromium.org/>
Si usas Firefox: descarga de <https://github.com/mozilla/geckodriver/releases>

1.2 Instalaci3n de Java

Si no tienes Java instalado:

WINDOWS: 1. Descarga JDK 11 LTS desde [oracle.com](https://www.oracle.com/in/java/technologies/javase-downloads.html) 2. Instala en: C:\Files 3. Agrega a PATH: - Abre Environment Variables - Agrega: C:\Files

MAC: brew install openjdk@11 sudo ln -sf /usr/local/opt/openjdk@11/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk-11.jdk

LINUX (Ubuntu/Debian): sudo apt-get update sudo apt-get install openjdk-11-jdk

1.3 Instalaci3n de Maven

WINDOWS: 1. Descarga Maven desde [maven.apache.org](https://maven.apache.org/download.cgi) 2. Extrae en: C:\Files. Agrega a PATH: C:\Files

MAC: brew install maven

LINUX: sudo apt-get install maven

1.4 Clonar el Repositorio

git clone <https://github.com/Grisel86/cryptocurrency-exchange-qa-framework.git>
cd cryptocurrency-exchange-qa-framework

1.5 Instalar Dependencias

mvn clean install

Esto descargará todas las librerías necesarias (Selenium, TestNG, REST Assured, etc).

Tiempo esperado: 2-5 minutos (depende de internet)

Si falla, intenta:

```
mvn clean install -U
```

2. EJECUTAR TESTS LOCALMENTE

2.1 Correr Todos los Tests

```
mvn clean test
```

Qu   hace: 1. Limpia build anterior (clean) 2. Compila c  digo (compile) 3. Ejecuta todos los tests (test) 4. Genera reportes

Tiempo esperado: 3-5 minutos

2.2 Correr Solo Tests de Smoke (R  pido)

```
mvn test -Dgroups=smoke
```

Tests incluidos: Cr  ticos, r  pidos, representan flujo principal Tiempo esperado: 1-2 minutos

2.3 Correr Solo Tests de Regresi  n

```
mvn test -Dgroups=regression
```

Tests incluidos: Validaciones adicionales, edge cases, escenarios

2.4 Correr Solo Tests de API

```
mvn test -Dgroups=api
```

Tests incluidos: REST API testing, sin Selenium

2.5 Correr Solo Tests de UI

```
mvn test -Dgroups=ui
```

Tests incluidos: Selenium browser automation

2.6 Correr una Clase Espec  fica

```
mvn test -Dtest=LoginTests
```

Corre solo LoginTests.java

2.7 Correr un Test Espec  fico

```
mvn test -Dtest=LoginTests#testLoginWithValidCredentials
```

Corre solo ese test

2.8 Correr con Firefox (en lugar de Chrome)

```
mvn test -Dbrowser=firefox
```

Por defecto usa Chrome. Si quieres Firefox:

2.9 Correr Tests en Paralelo

```
mvn test -T 1C
```

Ejecuta tests en paralelo (1 thread por core). M  s r  pido pero puede ser menos estable.

2.10 Modo Verbose (m  s logs)

```
mvn test -X
```

Muestra MUCHA informaci  n para debugging.

2.11 Saltar Tests (solo compilar)

```
mvn clean install -DskipTests
```

¿Stil si solo quieres revisar cdigo, no ejecutar.

3. ENTENDER LOS RESULTADOS

3.1 Salida Típica en Console

```
[INFO] Running com.trading.ui.LoginTests
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.23 s
[INFO]
[INFO] Running com.trading.api.TradingAPITests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.12 s
[INFO]
[INFO] BUILD SUCCESS
```

Significa: Todos los tests pasaron

3.2 Cuando Un Test Falla

```
[ERROR] FAILURE!
[ERROR] Test com.trading.ui.LoginTests.testLoginWithValidCredentials FAILED
[ERROR] Expected true but got false
[ERROR] at org.testng.Assert.assertTrue(Assert.java:41)
```

Significa: Un test falló, con descripción del error

3.3 Cuando Hay Error de Compilación

```
[ERROR] COMPILATION ERROR
[ERROR] ...
[ERROR] symbol: class SomeClass cannot be found
```

Significa: Hay un error en el cdigo Java, no en los tests

Solución: Revisa el import y la clase

3.4 Cuando Falla el Setup

```
[ERROR] BeforeClass: setUp
[ERROR] NoSuchSessionException: invalid session id
```

Significa: Problema al inicializar WebDriver

Posibles causas: - ChromeDriver no encontrado - Puerto ocupado - Permisos insuficientes

4. GITHUB ACTIONS PIPELINES

4.1 Entender los Workflows

Ubicación: .github/workflows/

Archivos principales: - ui-tests.yml: Corre tests de UI - api-tests.yml: Corre tests de API - all-tests.yml: Corre todos los tests

4.2 Cuando se Ejecutan los Pipelines

Los pipelines se ejecutan automáticamente cuando:

1. Haces PUSH a main o develop git push origin main
2. Creas un PULL REQUEST a main o develop (automáticamente antes de merge)

Puedes ver el estado en GitHub -> Actions

4.3 Configurar GitHub Actions

PASO 1: Ir a GitHub Settings del repo

Repo -> Settings -> Actions -> General

PASO 2: Dar permisos a GitHub Actions

Actions permissions: - Allow all actions and reusable workflows (recomendado)

Workflow permissions: - Read and write permissions - Allow GitHub Actions to create and approve pull requests

PASO 3: Agregar Secrets (si necesitas)

Settings -> Secrets -> New repository secret

Ejemplo: Si necesitas API token Name: API_TOKEN Value: tu-token-aqui

Luego en workflow: env: API_TOKEN: \${{ secrets.API_TOKEN }}

4.4 Ver Resultados de Pipelines

En GitHub:

1. Ir a Actions tab del repo
2. Ver lista de workflow runs
3. Haz click en un run para ver detalles
4. Ver logs de cada step
5. Descargar artifacts (reportes)

4.5 Interpretar Resultados de Pipeline

Status: Success (verde) Significa: Todos los tests pasaron, build OK

Status: Failure (rojo) Significa: Al menos un test falló o hay error de compilación

Status: Cancelled (gris) Significa: Workflow fue cancelado manualmente

4.6 Descargar Reportes del Pipeline

En Actions tab: 1. Click en el workflow run 2. Scroll down a "Artifacts" 3. Click en artifact name (ej: ui-test-reports) 4. Se descarga ZIP con reportes

Dentro está: index.html - abre en browser

4.7 Modificar un Workflow

Ejemplo: Cambiar qué tests se corren

Archivo: .github/workflows/ui-tests.yml

```
- name: Run UI Tests
  run: mvn clean test -Dgroups=smoke
```

Cambiar a:

```
- name: Run UI Tests
  run: mvn clean test -Dgroups=ui,smoke
```

Luego: git add .github/workflows/ui-tests.yml git commit -m "Update: run ui and smoke tests" git push

4.8 Crear un Nuevo Workflow

PASO 1: Crear archivo

.github/workflows/performance-tests.yml

PASO 2: Agregar contenido

name: Performance Tests

on: [push, pull_request]

```
jobs:
  perf-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up JDK
        uses: actions/setup-java@v3
        with:
          java-version: 11
          distribution: 'temurin'
          cache: maven

      - name: Run Performance Tests
        run: mvn test -Dgroups=performance

      - name: Upload reports
        if: always()
```

```
uses: actions/upload-artifact@v3
with:
  name: perf-reports
  path: target/surefire-reports/
```

PASO 3: Commit y push

git add .github/workflows/performance-tests.yml git commit -m "Add performance test workflow" git push

5. DEBUGGING Y TROUBLESHOOTING

5.1 Tests Fallan Localmente

PROBLEMA: Tests pasan en CI pero fallan localmente

SOLUCION: 1. Borra Maven cache: mvn clean

2. Actualiza dependencias: mvn clean install -U
3. Verifica Java version: java -version
4. Verifica que puedas acceder a URLs de test: ping www.saucedemo.com

5.2 WebDriver No Se Encuentra

ERROR: ChromeDriver not found

SOLUCION:

Opci3n 1: El c3digo deber3a auto-descargar (WebDriverManager)
Verifica que tengas internet

Opci3n 2: Descarga manual
Descarga chromedriver de: <https://chromedriver.chromium.org/>
Pon en: /path/to/project/drivers/

5.3 Elemento No Encontrado (NoSuchElementException)

Exception: NoSuchElementException
Element not found with locator: By.id("login-button")

DEBUGGING: 1. Abre Chrome DevTools (F12) 2. Usa console para verificar elemento:
document.getElementById("login-button")

3. Si no aparece:
 - Elemento est3 en iframe
 - Elemento se carga con JavaScript (agregar waits)
 - Locator est3 incorrecto

SOLUCION:

```
// Agregar explicit wait
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.elementToBeClickable(By.id("login-button")));
```

5.4 Tests Lentos

PROBLEMA: Tests toman mucho tiempo

SOLUCION: 1. Reduce timeouts (si es seguro): protected static final int TIMEOUT_SECONDS = 5;

2. Ejecuta en paralelo: mvn test -T 1C
3. Agrega solo waits expl3citos (no impl3citos): Solo WebDriverWait, no implicitlyWait
4. Usa operaciones m3s eficientes: // MAL: Thread.sleep(5000);

// BIEN: wait.until(ExpectedConditions.textToBePresentInElement(element, "texto"));

5.5 Tests Aleatorios (Flakiness)

PROBLEMA: Tests a veces pasan, a veces fallan

CAUSA: Problemas de sincronizaci3n

SOLUCION:

```
// MALO (implicit wait):
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
driver.findElement(By.id("element")).click();

// BUENO (explicit wait):
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.elementToBeClickable(By.id("element"))).click();
```

Siempre usa Explicit Waits

5.6 Tests Fallan Solo en GitHub Actions

PROBLEMA: Funciona localmente pero falla en CI/CD

CAUSA: Diferencias entre local y CI

SOLUCION: 1. CI usa headless browser: En local: browser visible En CI: browser sin UI

2. Agrega flag headless si necesitas:

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless");
driver = new ChromeDriver(options);
```

3. Aumenta timeouts en CI: protected static final int TIMEOUT_SECONDS = 15;

4. Revisa logs del pipeline para más detalles

5.7 Port Already in Use

ERROR: Address already in use: bind

SOLUCION:

```
# Encuentra qué está usando el puerto
lsof -i :8080 # en Mac/Linux
netstat -ano | findstr :8080 # en Windows
```

```
# Mata el proceso (si es seguro)
kill -9 <PID> # Mac/Linux
taskkill /PID <PID> /F # Windows
```

```
# O simplemente reinicia la máquina
```

5.8 Timeout Esperando Elemento

ERROR: TimeoutException waiting for element

RAZON: Elemento toma más tiempo en cargar

SOLUCION: 1. Aumenta timeout: new WebDriverWait(driver, Duration.ofSeconds(20))

2. Verifica que el elemento está visible:

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("element")))
```

3. Si está en iframe:

```
driver.switchTo().frame("frameId");
driver.findElement(By.id("element"));
driver.switchTo().defaultContent();
```

6. VER REPORTES Y BUGS

6.1 Ubicación de Reportes

Después de ejecutar tests:

target/surefire-reports/

Archivos principales: - index.html: Reporte HTML (abre en browser) - TEST-com.trading.ui.LoginTests.xml: Resultados en XML - com.trading.ui.LoginTests.txt: Logs en texto

6.2 Abrir Reporte HTML

```
cd target/surefire-reports/
open index.html # Mac
xdg-open index.html # Linux
```

start index.html # Windows

O simplemente arrastra el archivo al browser

6.3 Entender el Reporte

En index.html verÃs:

- Test Summary: Total Tests: 15 Passed: 12 Failed: 3 Skipped: 0
- Detalles por Test: Test Name: testLoginWithValidCredentials Status: PASS Duration: 2.34 seconds
- Tests Fallidos: Nombre del test Error message exacto Stack trace (lÃnea donde fallÃ³)

6.4 Analizar un Test Fallido

En el reporte, haz click en test fallido:

VerÃs: 1. DescripciÃ³n del test 2. Mensaje de error 3. Stack trace 4. MÃ©todo y lÃnea exacta donde fallÃ³

Ejemplo:

```
testLoginWithValidCredentials FAILED
Expected true but was false
at org.testng.Assert.assertTrue(LoginTests.java:45)
```

Significa: LÃnea 45 de LoginTests.java fallÃ³

Abre LoginTests.java lÃnea 45 y revisa

6.5 Agregar Screenshots en Caso de Fallo

Modifica BaseTest.java para capturar screenshots:

```
@AfterClass(alwaysRun = true)
public void tearDown() {
    if (driver != null) {
        try {
            // Si test fallÃ³, captura screenshot
            File screenshot = ((TakesScreenshot) driver)
                .getScreenshotAs(OutputFormat.FILE);
            String timestamp = System.currentTimeMillis() + "";
            FileUtils.copyFile(screenshot,
                new File("target/screenshots/screenshot_" + timestamp + ".png"));
        } catch (Exception e) {
            e.printStackTrace();
        }
        driver.quit();
    }
}
```

Necesitas: org.apache.commons:commons-io:1.3.2 en pom.xml

6.6 Ver Logs Detallados

Si agregas logging en tests:

```
@Test
public void testLogin() {
    System.out.println("Starting login test");
    LoginPage loginPage = new LoginPage(driver);
    System.out.println("LoginPage created");

    loginPage.enterUsername("user");
    System.out.println("Username entered");

    DashboardPage dashboard = loginPage.clickLogin();
    System.out.println("Login button clicked");
}
```

Los logs aparecen en: - Console (cuando corres localmente) - Reporte artifact en GitHub Actions

7. MANTENIMIENTO DEL FRAMEWORK

7.1 Actualizar Dependencias

PeriÃdicamente (mensualmente):

```
# Ver qu  dependencias tienen updates
mvn versions:display-dependency-updates
```

```
# Actualizar autom ticamente (con cuidado)
mvn versions:use-latest-versions
```

Luego:

```
git add pom.xml
git commit -m "Update dependencies"
git push
```

7.2 Agregar Nuevos Tests

ESTRUCTURA: 1. Crea test class en src/test/java/com/trading// 2. Extiende BaseTest o APIBaseTest 3. Agrega @Test annotations 4. Define @Test(groups = {"!})

EJEMPLO:

```
package com.trading.ui;

import com.trading.base.BaseTest;
import org.testng.annotations.Test;

public class NewFeatureTests extends BaseTest {

    @Test(description = "Test new feature", groups = {"smoke"})
    public void testNewFeature() {
        // Test code
    }
}
```

7.3 Refactorizar Locators

Si UI cambia y locators fallan:

1. Actualiza en Page Object:

```
// ANTES:
private By loginButton = By.id("login-button");

// DESPUES (si ID cambi ):
private By loginButton = By.xpath("//button[contains(text(), 'Sign In')]");
```

2. Verifica en Chrome DevTools que funcione
3. Commit y push
4. Tests deber an pasar nuevamente

7.4 Documentar Cambios

Cuando modifiques framework:

```
git commit -m "Update: Description of change"
```

Ejemplo:

```
git commit -m "Add: New security tests for OWASP"
git commit -m "Fix: Flaky login test with explicit waits"
git commit -m "Update: Selenium version to 4.20"
```

7.5 Limpiar Tests Obsoletos

Si un test ya no es relevante:

```
# Opci n 1: Marcar como skip (mantener c digo)
@Test(skip = true, description = "Old feature - no longer tested")
public void testOldFeature() { }
```

```
# Opci n 2: Eliminar completamente
git rm src/test/java/com/trading/ui/OldTests.java
git commit -m "Remove: Obsolete tests"
```

7.6 Mantenimiento de Page Objects

Cada 2-3 semanas, revisa Page Objects:

1. Verifica que todos los locators siguen siendo v lidos
2. Si UI cambi , actualiza locators
3. Consolidar m todos duplicados
4. Mejorar documentaci n

7.7 Revisar y Limpiar Logs

Despu s de muchas ejecuciones:

```
# Ver tama o de target/  
du -sh target/
```

```
# Limpiar build antiguo  
mvn clean
```

```
# Limpiar solo logs  
rm -rf target/logs/*
```

8. BEST PRACTICES

8.1 Commits Bien Hechos

MALO: `git commit -m " stuff "` `git commit -m " fix "` `git commit -m " update "`

BUENO: `git commit -m " Add: Login tests with valid credentials "` `git commit -m " Fix: Flaky dashboard test with explicit waits "` `git commit -m " Update: Selenium from 4.15 to 4.20 "`

Template: [Type]: Description

Types: - Add: Nuevo c digo - Fix: Correcci n de bug - Update: Actualizaci n de dependencia - Refactor: Reorganizaci n de c digo - Remove: Eliminar c digo

8.2 Nombres de Tests Descriptivos

MALO: `public void test1() { }` `public void loginTest() { }`

BUENO: `public void testLoginWithValidCredentials() { }` `public void testLoginFailsWithInvalidPassword() { }` `public void testLoginFailsWithoutUsername() { }`

Siempre: verbo + qu  se prueba + condici n

8.3 Usar @Test Description

```
@Test(  
    description = "User can successfully login with valid credentials",  
    groups = {"smoke"}  
)  
public void testLoginWithValidCredentials() { }
```

Esta descripci n aparece en reportes

8.4 Usar Explicit Waits Siempre

```
// NUNCA:  
Thread.sleep(5000);
```

```
// NUNCA:  
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

```
// SIEMPRE:  
wait.until(ExpectedConditions.elementToBeClickable(By.id("button")));
```

8.5 Separar Setup de Acciones

```
@BeforeMethod  
public void loginBeforeTest() {  
    User user = new User("user", "pass");  
    LoginPage loginPage = new LoginPage(driver);  
    dashboardPage = loginPage.login(user.getUsername(), user.getPassword());  
}
```

```
@Test  
public void testMainFeature() {  
    // Aqu  ya estamos logged in  
    // Solo testea la feature  
}
```

8.6 Un Assert por Test (cuando sea posible)

```
// MEJOR:  
@Test  
public void testLoginSuccessful() {
```

```
// ... login code
Assert.assertTrue(dashboardPage.isDashboardDisplayed());
}

// ACCEPTABLE (si son relacionados):
@Test
public void testProductsDisplayCorrectly() {
    Assert.assertEquals(dashboardPage.getProductCount(), 6);
    Assert.assertTrue(dashboardPage.isProductsDisplayed());
}
```

9. FAQ

P: ¿Cuántos tests deberÃa tener? R: MÃnimo 10-15 por tipo. Para portfolio: 30+ tests total es impresionante.

P: ¿Con quÃ© frecuencia ejecuto tests? R: En local: antes de cada push. En CI: automÃticamente en cada push.

P: ¿QuÃ© pasa si un test falla en CI pero funciona localmente? R: Aumenta timeouts, revisa logs del pipeline, intenta reproducir localmente.

P: ¿Puedo skippear un test? R: SÃ, usa @Test(skip = true), pero es temporal. ArrÃglalo eventualmente.

P: ¿CÃmo manejo test data? R: En src/test/resources/data/. O usa factory methods en tests.

P: ¿CÃmo aseguro que tests no sean flaky? R: Usa Explicit Waits, no duermas threads, test data consistente.

P: ¿Necesito todos los browsers? R: Para portfolio: Chrome es suficiente. Muestra Firefox como opciÃn.

P: ¿CÃmo veo si un test es rÃpido o lento? R: En reporte: cada test muestra duration. <3 sec es rÃpido, >10 sec es lento.

P: ¿Puedo correr tests en paralelo? R: SÃ, pero arriesgado si tests comparten estado. Usa thread-count=3 mÃximo.

P: ¿QuÃ© hago si tengo 100+ tests? R: Organiza en grupos (smoke, regression, api). Corre smoke en cada push, full nightly.

RESUMEN

LOCAL TESTING: mvn clean test # Todo mvn test -Dgroups=smoke # RÃpido

GITHUB ACTIONS: Push a main/develop -> se ejecutan automÃticamente Ver resultados en GitHub Actions tab

DEBUGGING: Revisa reportes en target/surefire-reports/index.html Agrega logs y screenshots

MANTENIMIENTO: Actualiza dependencias mensualmente Refactoriza locators si UI cambia Agrega tests para nuevas features

ÃListo para correr!